

Google Cloud Concepts Used in This Deployment

Reference glossary of every Google Cloud concept/product touched while deploying QuizzDjangoForms to Cloud Run, in the order they were encountered.

1 Google Cloud SDK (`gcloud` CLI)

The command-line tool used to manage all Google Cloud resources — authentication, projects, billing, Cloud SQL, Secret Manager, Cloud Run, etc.

Used for: every command in this deployment was run through `gcloud`.

Documentation: <https://cloud.google.com/sdk/docs/overview>

2 Authentication & Accounts (`gcloud auth`)

Controls which Google identity (user account or service account) `gcloud` acts as, and which one is "active" for subsequent commands.

Used for: switching from the work account (`afinder@mozilla.com`) to the personal account (`alex.finder.89@gmail.com`) via `gcloud auth login` and `gcloud config set account`.

Documentation: <https://cloud.google.com/sdk/gcloud/reference/auth>

3 Billing Accounts

A billing account links a payment method to one or more projects. A project cannot use most paid services (Cloud Run, Cloud SQL, etc.) until it is linked to an **open** billing account.

Used for: checking billing account status with `gcloud billing accounts list` and linking the new project with `gcloud billing projects link`.

Documentation: <https://cloud.google.com/billing/docs/how-to/manage-billing-account>

4 Projects (Resource Manager)

A project is the top-level container for Google Cloud resources — it groups billing, IAM permissions, APIs, and all created resources (Cloud Run services, databases, secrets, etc.) under one ID.

Used for: created the `quizz-django-forms` project with `gcloud projects create` to host all resources for this app.

Documentation: <https://cloud.google.com/resource-manager/docs/creating-managing-projects>

5 Identity and Access Management (IAM)

Controls *who* (users, groups, or service accounts) can do *what* (roles/permissions) on *which* resources. Every Google Cloud resource access decision goes through IAM.

Used for: granting the Cloud Run service's default **Compute Engine service account** the `roles/secretmanager.secretAccessor` role so it could read secrets at runtime (`gcloud projects add-iam-policy-binding`).

Documentation: <https://cloud.google.com/iam/docs/overview>

6 Enabling APIs (Service Usage)

Each Google Cloud product exposes its functionality as an API that must be explicitly enabled per project before it can be used.

Used for: enabling `run` , `sqladmin` , `artifactregistry` , `cloudbuild` , `secretmanager` , and `compute` APIs via `gcloud services enable` .

Documentation: <https://cloud.google.com/service-usage/docs/enable-disable>

7 Cloud SQL for PostgreSQL

A fully managed relational database service. Provides a persistent PostgreSQL instance so application data survives container restarts and redeploys (unlike SQLite on Cloud Run's ephemeral filesystem).

Used for: created the `quizz-db` instance (Postgres 16, `db-f1-micro` tier), plus the `quizzdb` database and `quizzuser` user that the Django app connects to.

Documentation: <https://cloud.google.com/sql/docs/postgres/create-instance>

8 Secret Manager

A secure store for API keys, passwords, and other sensitive configuration. Secrets are versioned and access is controlled via IAM, rather than being baked into source code or images.

Used for: storing the Django `SECRET_KEY` , the Cloud SQL database password, and the Django admin password, then injecting them into Cloud Run as environment variables via `--set-secrets` .

Documentation: <https://cloud.google.com/secret-manager/docs/overview>

9 Artifact Registry

A managed repository for container images (and other build artifacts). Cloud Run pulls the application's container image from here.

Used for: the `cloud-run-source-deploy` Docker repository was auto-created in `europa-west1` to store the built `quizz-django-forms` image.

Documentation: <https://cloud.google.com/artifact-registry/docs/overview>

10 Cloud Build

A managed CI/CD service that builds artifacts — in this case, building a Docker image from a `Dockerfile` and pushing it to Artifact Registry.

Used for: `gcloud run deploy --source .` automatically uploaded the source and ran a Cloud Build job to build the container image.

Documentation: <https://cloud.google.com/build/docs/overview>

11 Cloud Run (Services)

A serverless platform for running stateless containers. Automatically scales (including to zero) based on incoming HTTP traffic, and provides a public HTTPS URL.

Used for: hosting the Django app itself — the `quizz-django-forms` service, deployed via `gcloud run deploy`, running `gunicorn` behind Google's managed HTTPS load balancer.

Documentation: <https://cloud.google.com/run/docs/overview/what-is-cloud-run>

12 Connecting Cloud Run to Cloud SQL

Cloud Run can connect to a Cloud SQL instance through a secure Unix domain socket (`/cloudsql/<CONNECTION_NAME>`) without needing a public IP or manually managing the Cloud SQL Auth Proxy.

Used for: the `--add-cloudsql-instances` flag attached `quizz-django-forms:europa-west1:quizz-db` to the Cloud Run service, and Django's `DB_HOST` env var pointed to the corresponding socket path.

Documentation: <https://cloud.google.com/sql/docs/postgres/connect-run>

13 Cloud Run Jobs

Unlike Cloud Run *services* (which respond to HTTP requests), Cloud Run *jobs* run a container to completion for a finite task and then exit — ideal for one-off scripts.

Used for: the `create-superuser` job ran `python manage.py createsuperuser --noinput` once against the production database, then was deleted.

Documentation: <https://cloud.google.com/run/docs/create-jobs>

Generated for the QuizzDjangoForms project — companion reference to the Cloud Run deployment tutorial, 2026-06-11.